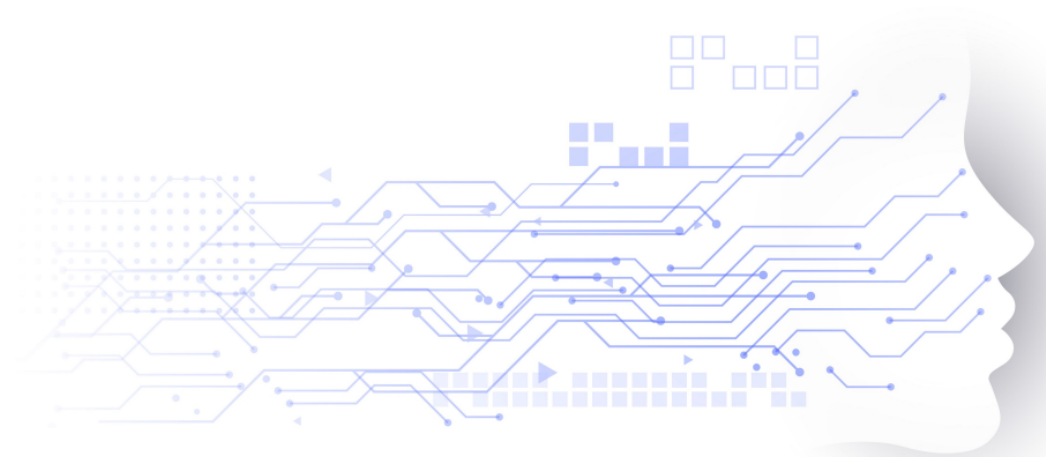




Neural Networks

COSC2753 Machine Learning

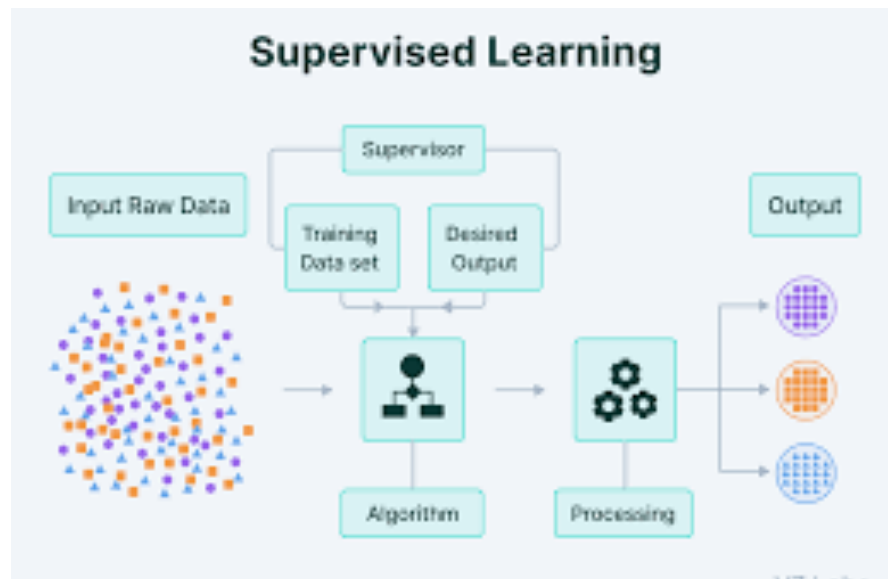
Semester 2, 2026



Content

- Biologically inspired learning – perceptron
- Multi-layer perceptron
- Back propagation
- Practical issues

Revision: Supervised Learning



To find the “best” hypothesis: $h(\mathbf{x}) \approx f(\mathbf{x})$

Experience: From which to learn (or find) the hypothesis

Task: Learn a model that to predict new “unseen” inputs.

Performance: how closely the hypothesis predicts the output

Algorithms:

Linear Regression, Polynomial Regression

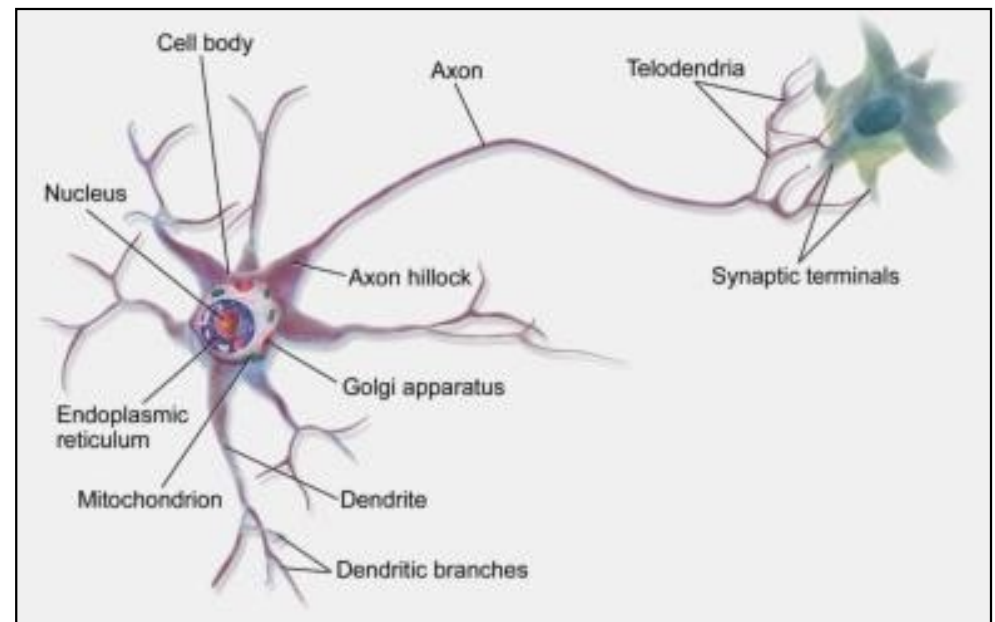
Logistics Regression, Decision Trees

Random Forest

Perceptron

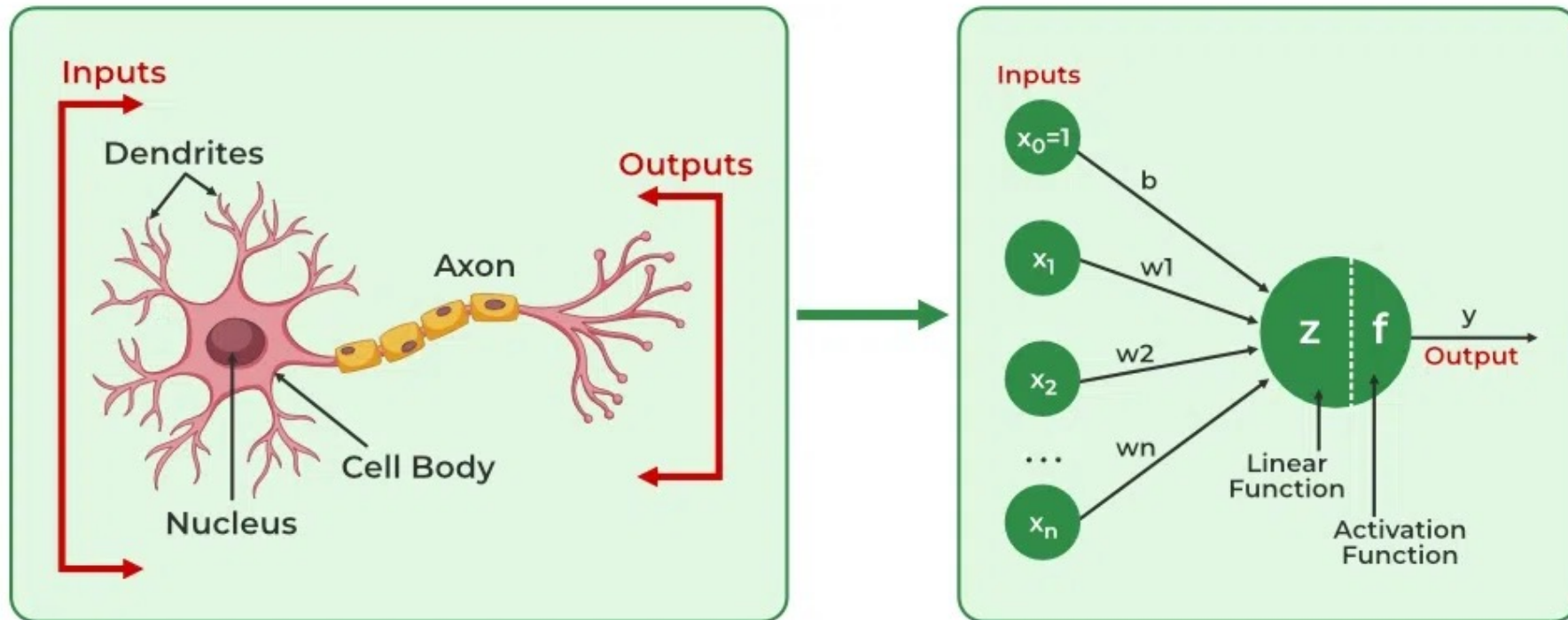
Biologically Inspired Learning

- Neural Networks inspired by the structure of human brain
 - *many ML techniques are inspired by biology, or methods that humans and animals take for learning*
- A Neuron is formed of:
 - A series of incoming synapses
 - An activation cell
 - A single outgoing synapse connecting to other neurons



Perceptron

A Neuron is modelled as a *Perceptron* (Rosenblatt 1962):



$$z = b + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

$$y = f(z) = f(b + w_1x_1 + w_2x_2 + \dots + w_nx_n)$$

Perceptron

- A Neuron is modeled as a **Perceptron** (Rosenblatt 1962):
- A Perceptron consists of:

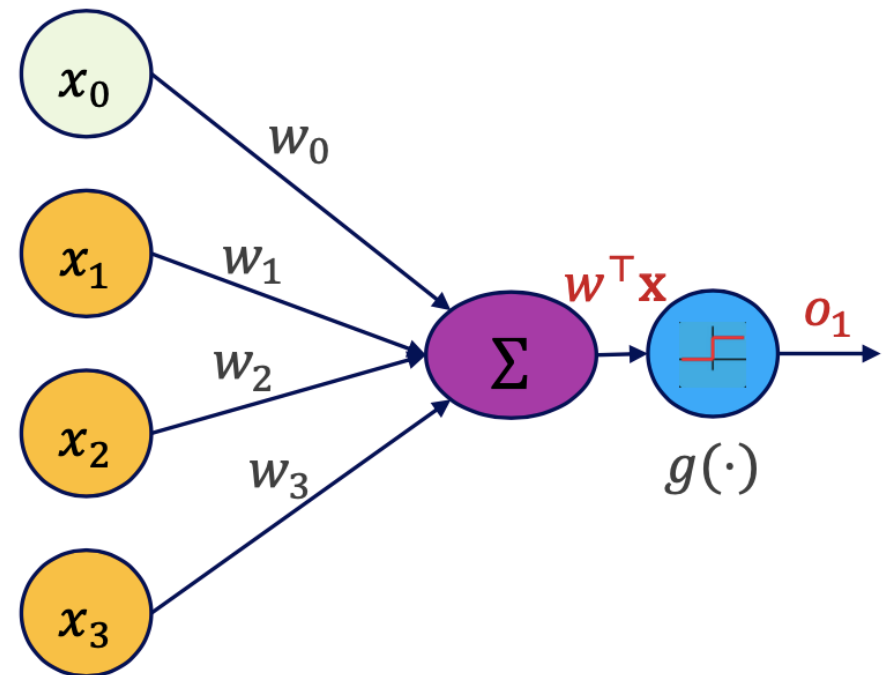
Multiple Input Connections: $\mathbf{x} = [x_0, x_1, \dots, x_m]^T$

Bias (additional input): $x_0 = 1$

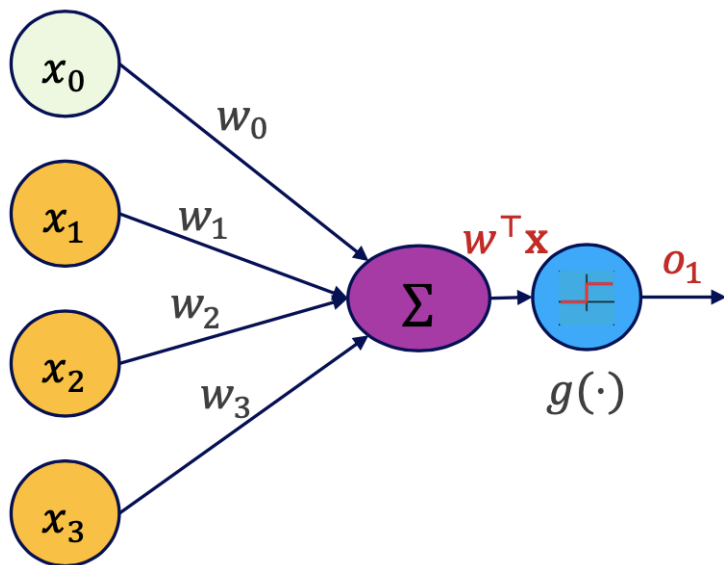
Weights on each input: $\mathbf{w} = [w_0, w_1, \dots, w_m]^T$

Activation Function: $g(\mathbf{w}^T \mathbf{x})$

Single output: o_1



Perceptron



The output of the perceptron is calculated by:

- Summing the products of each input and their weights

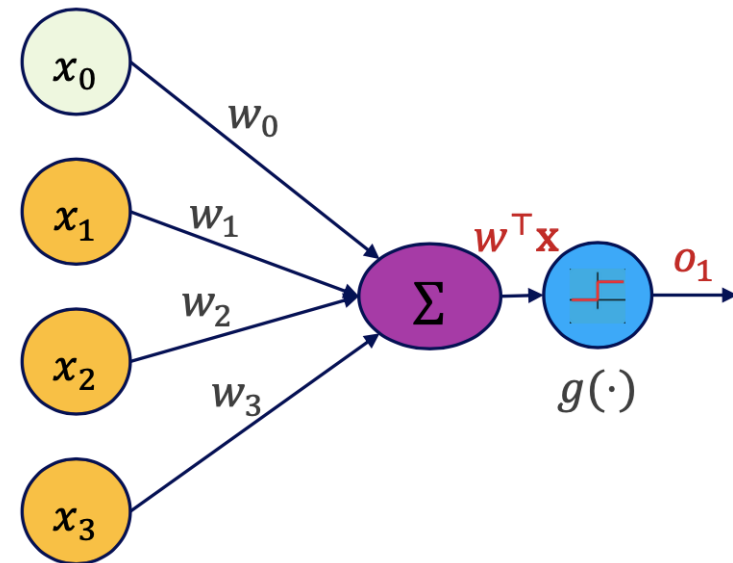
$$w_0x_0 + w_1x_1 + w_2x_2 + \dots = \sum_{i=0}^m w_i x_i = \mathbf{w}^T \mathbf{x}$$

- Passing it through the activation function
- A simple step function is used:

$$o_1 = g(\mathbf{w}^T \mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} \geq 0 \\ 0 & \text{if } \mathbf{w}^T \mathbf{x} < 0 \end{cases}$$

Perceptron as a Binary Classifier

- The perceptron can function as a binary classifier
- For example, two classes: (+, -)
 - Output value of 1, means class +
 - Output value of 0, means class -

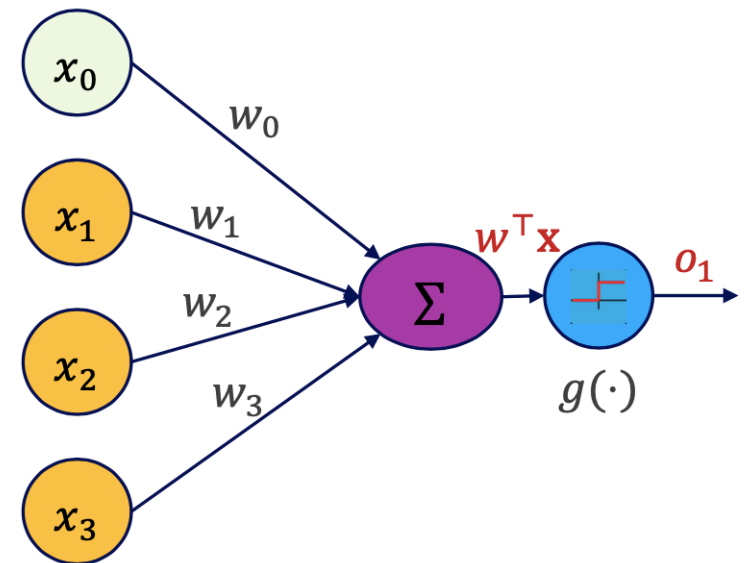


Learning with a Perceptron

- Learning with a Perceptron involves finding values for the **weights**:
 - Observe that the calculation essentially linear regression!

$$\sum_{i=0}^m w_i x_i = \mathbf{w}^T \mathbf{x}$$

- Thus we could use gradient descent!



Learning with a Perceptron

Simple update rule for greedy algorithm:

1. Initialize the weights (random, all zero, etc.)

$x(i)$

2. Repeat until convergence

For each example i in training dataset D

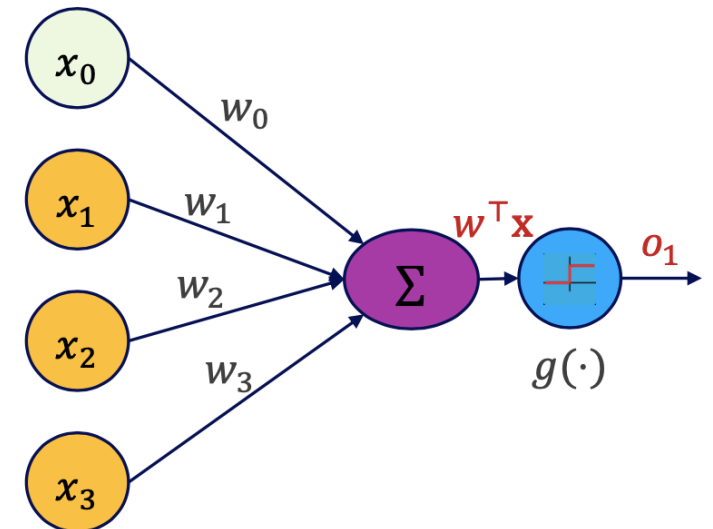
Calculate predicted output

$$o_1^{(i)} = g(\mathbf{w}^T \mathbf{x}^{(i)})$$

Update the weights

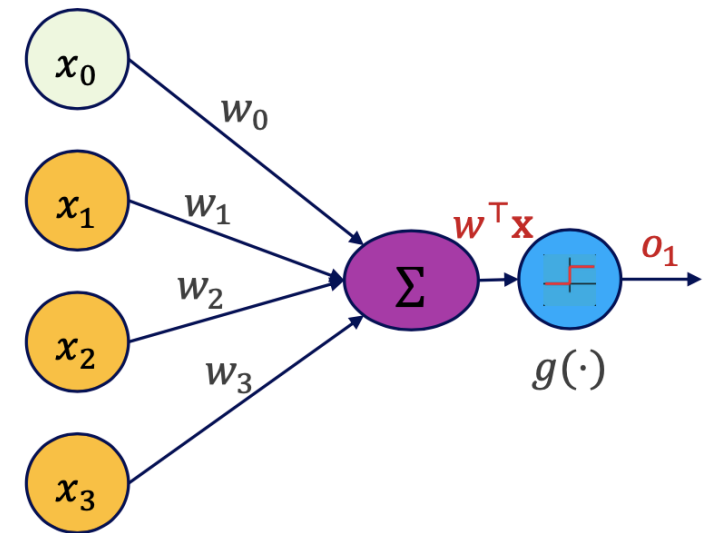
$$w_j = w_j + \alpha (y^{(i)} - o_1^{(i)}) x_j^{(i)}$$

Also uses the **learning rate**: $0 \leq \alpha \leq 1$



Learning with a Perceptron

- Perceptrons can model both **Regression** and **Classification**
 - This is dependent on the **Activation Function**
 - For the purpose of this lecture, we will focus on Classification
- Observe that a Perceptron is essentially
 - Linear Regression
 - Where the regression output is passed through the activation function!



$$w_0x_0 + w_1x_1 + w_2x_2 + \dots = \sum_{i=0}^m w_i x_i = \mathbf{w}^T \mathbf{x}$$

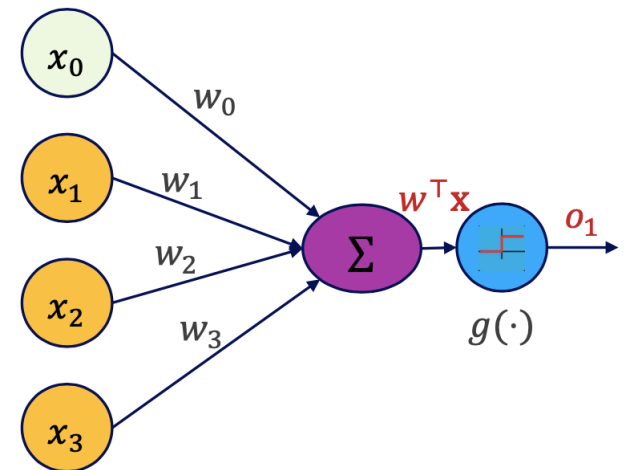
$$o = g(z) = g(w^T x)$$

Learning with a Perceptron

- We also observe that the basic perceptron is similar to **logistics regression**!
- Replace step function with **sigmoid** function

$$g(\mathbf{w}^T \mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T \mathbf{x}}} = \sigma(\mathbf{w}^T \mathbf{x})$$

- Now, learning weights is similar to gradient descent



Learning with Perceptron – Mini batch

- For Neural Networks, the learning is similar, except
 - Use **online learning**, where we don't calculate errors for all examples in the data set
 - Instead, we calculate the errors for a random **batch** of training example at a time

$$J(w) = \frac{1}{n} \sum_{i=1}^n \text{loss}(h_w(\mathbf{x}^{(i)}), y^{(i)}) = E_{x \sim p(x)} [\text{loss}(h_w(\mathbf{x}^{(i)}), y^{(i)})]$$

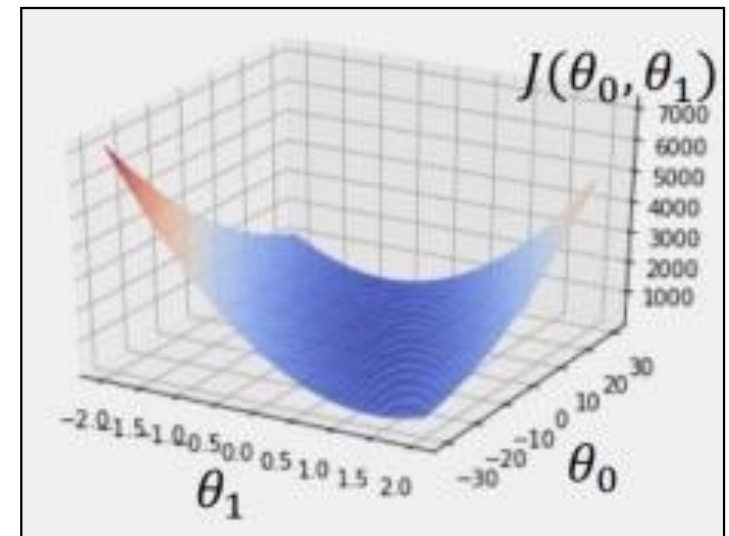
Learning with Perceptron – Mini batch

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \text{loss}(h_{\mathbf{w}}(\mathbf{x}^{(i)}), y^{(i)}) = E_{\mathbf{x} \sim p(\mathbf{x})} [\text{loss}(h_{\mathbf{w}}(\mathbf{x}^{(i)}), y^{(i)})]$$

$$E(\mathbf{w}) \equiv \frac{1}{b} \sum_{\mathbf{x}^{(i)} \sim p(\mathbf{x})} \text{loss}(y^{(i)}, h_{\mathbf{w}}(\mathbf{x}^{(i)}))$$

We only use **b samples** to compute the gradient descent update

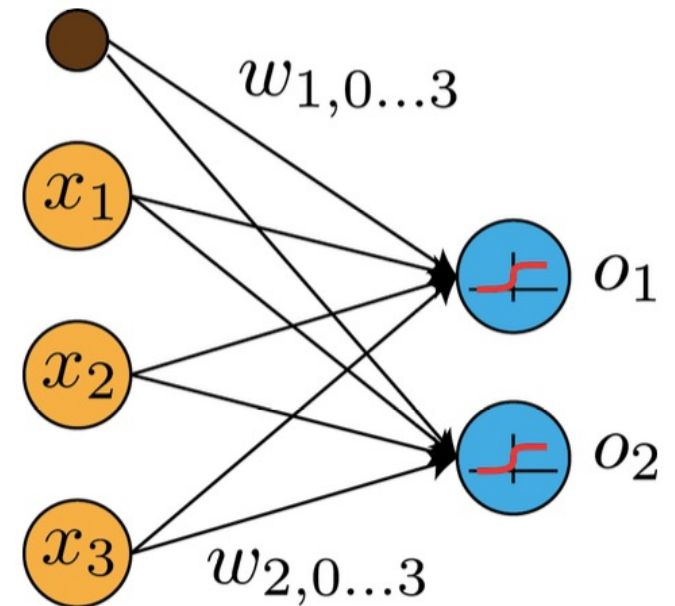
$$w_j = w_j - \alpha \frac{\delta}{\delta w_j} E(\mathbf{w})$$



Stochastic gradient descent

More than one Perceptron

- A single perceptron is effectively just a single logistics regression equation
 - For classification this is a binary classifier
 - But what about being more general?
- We can also represent a binary classifier using two perceptrons, connected to each output
 - Each output is: $o_j = \sigma(\mathbf{w}^T \mathbf{x})$
 - For **class +**, $o_1 = 1, o_2 = 0$
 - Vice versa for **class -**, $o_1 = 0, o_2 = 1$
 - To classify choose: $\max o_j$

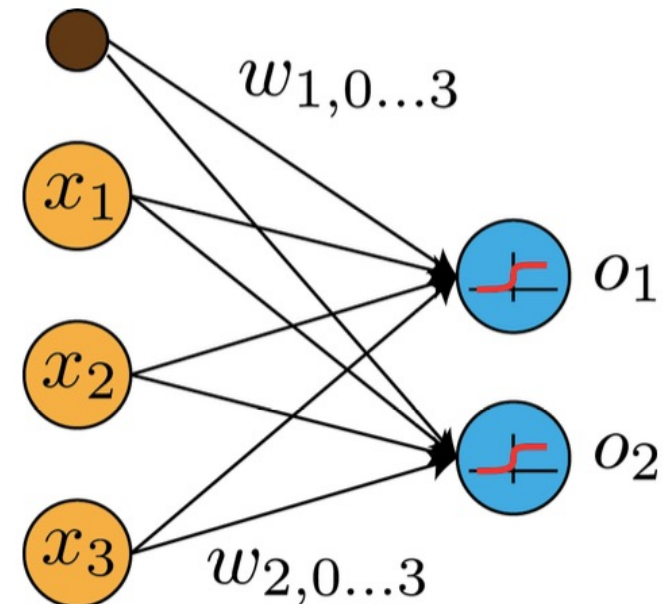


More than one Perceptron

Training is identical to with a single perceptron

$$\mathbf{w}^T = \begin{bmatrix} w_{1,0} & w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,0} & w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} = \begin{bmatrix} w_1^T \\ w_2^T \end{bmatrix}$$

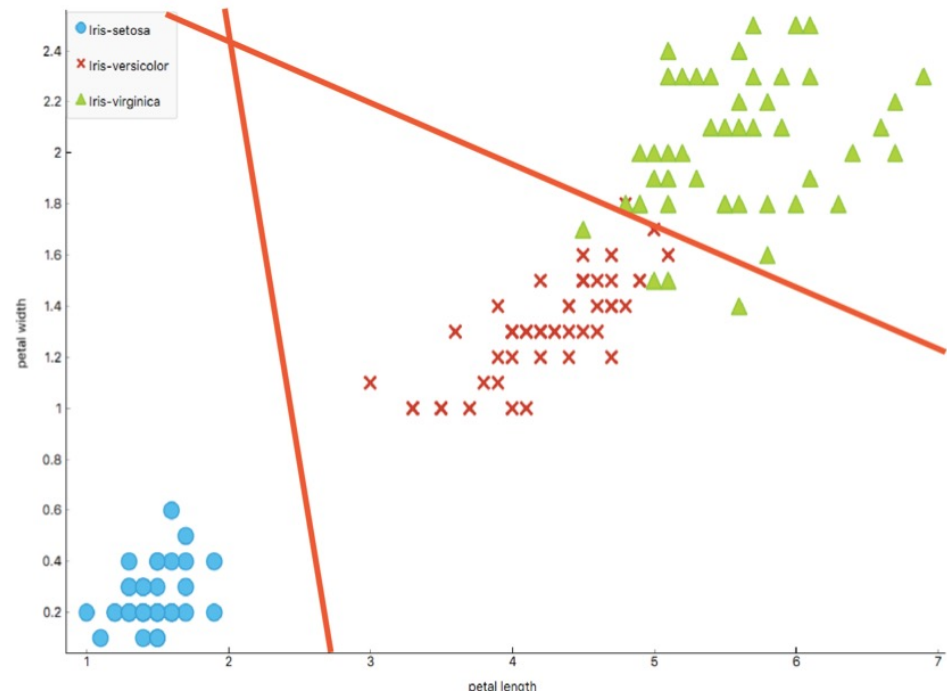
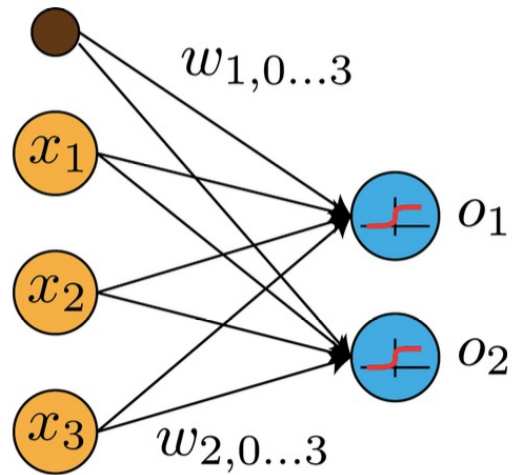
$$\begin{bmatrix} o_1 \\ o_2 \end{bmatrix} = \sigma \left(\begin{bmatrix} w_{1,0} & w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,0} & w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \right)$$



More than one Perceptron

Multiple classes: adding additional perceptrons, one for each class

→ multiple logistics regression functions



Multi-layer Perceptron

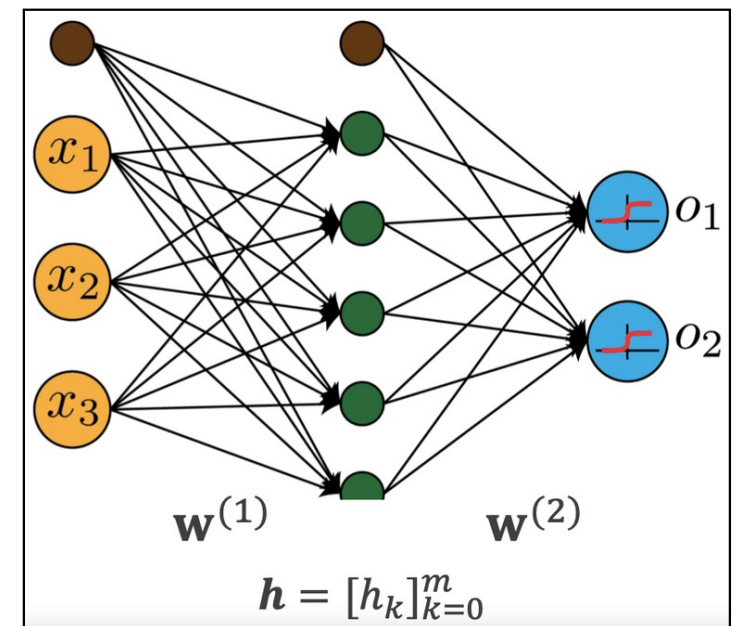
Multi-Layer Perceptron (MLP)

A **multi-layer perceptron**, introduces a **hidden layer** of perceptrons

- Each hidden layer perceptron has a sigmoid function
- The perceptrons are fully connected
- Using a feed-forward method

- $h_k = \sigma \left(\left(w_k^{(1)} \right)^\top \mathbf{x} \right)$
- $o_j = \sigma \left(\left(w_j^{(2)} \right)^\top \mathbf{h} \right)$

- Known as an **Artificial Neural Network (ANN)**

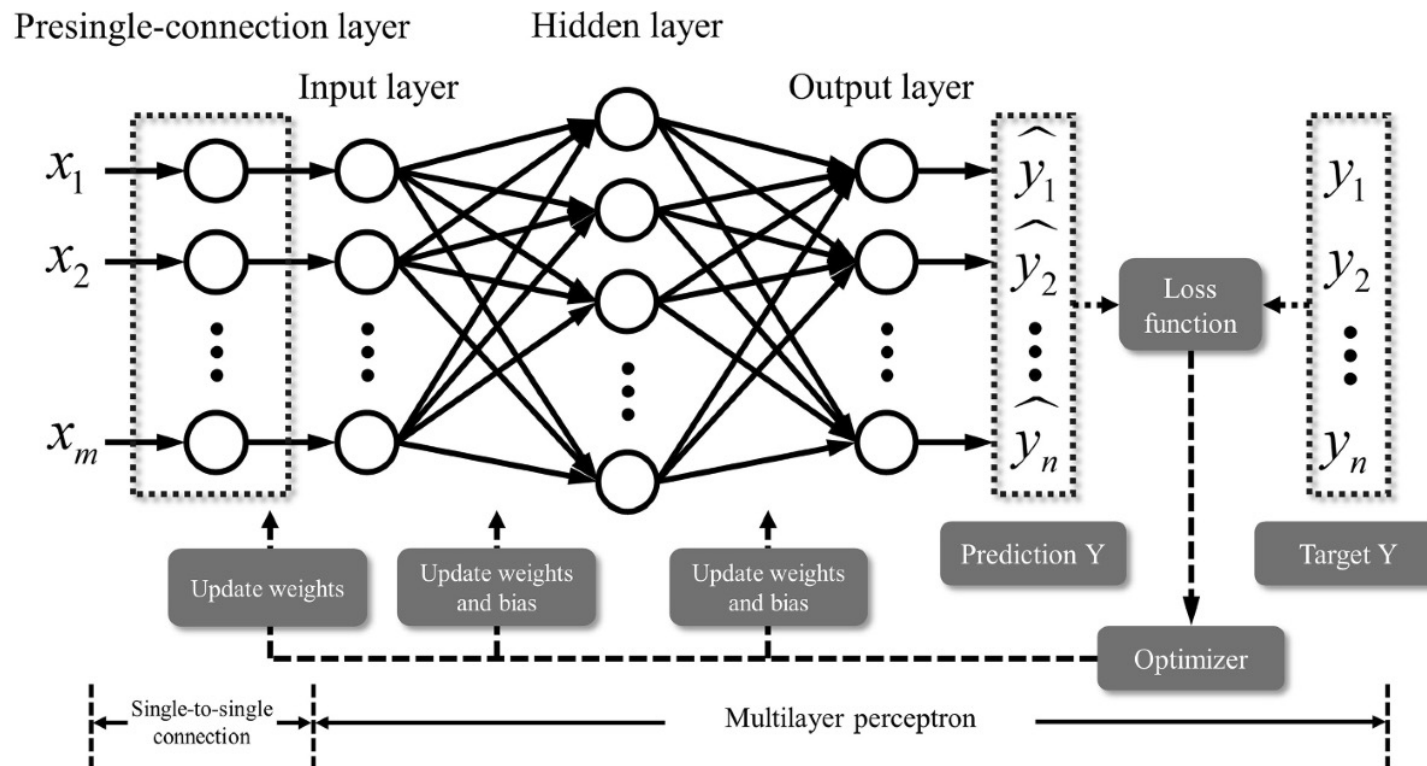


Training an MLP

Training a MLP

- How to train MLP: adapt gradient descent
- If we consider just the output layer
 - Loss function (error) is still between expected output and actual output

$$w_j^{(2)} = w_j^{(2)} + \alpha \sum_i (y^{(i)} - o_j^{(i)}) h_k^{(i)}$$



Training a MLP

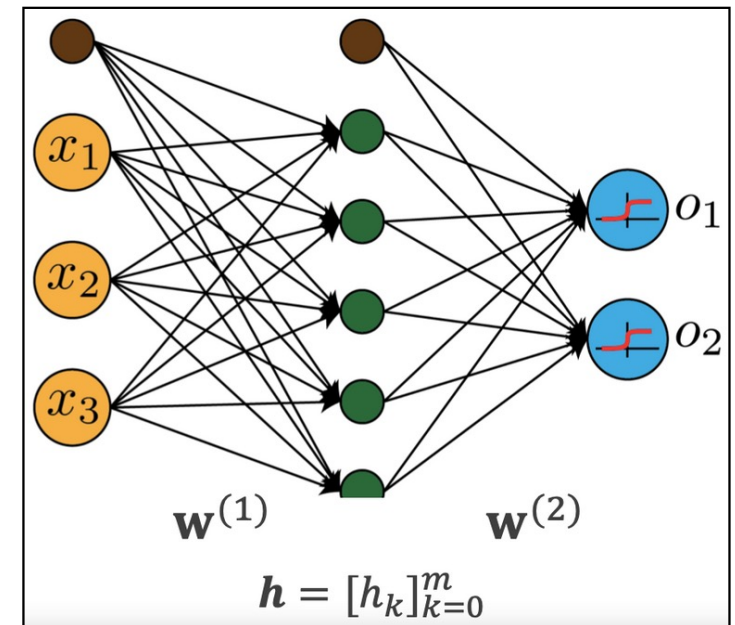
How can we train the **weights of the hidden layer**?

$$w_j^{(1)} = w_j^{(1)} + \alpha \sum_i \left(?^{(i)} - h_k^{(i)} \right) x_l^{(i)}$$

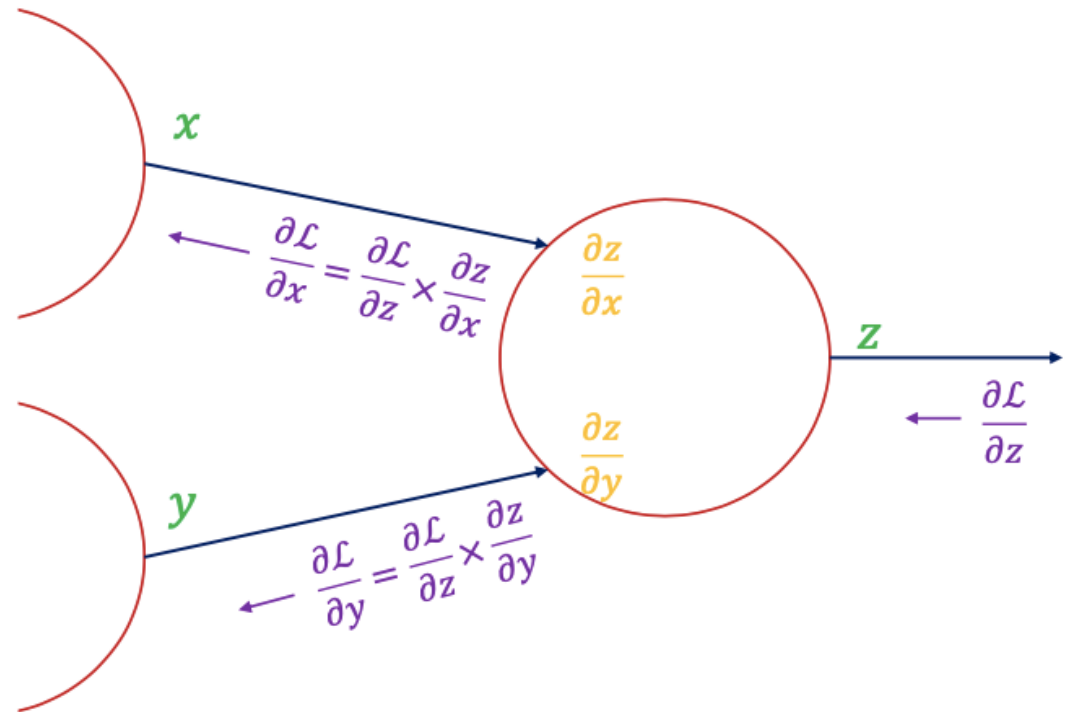
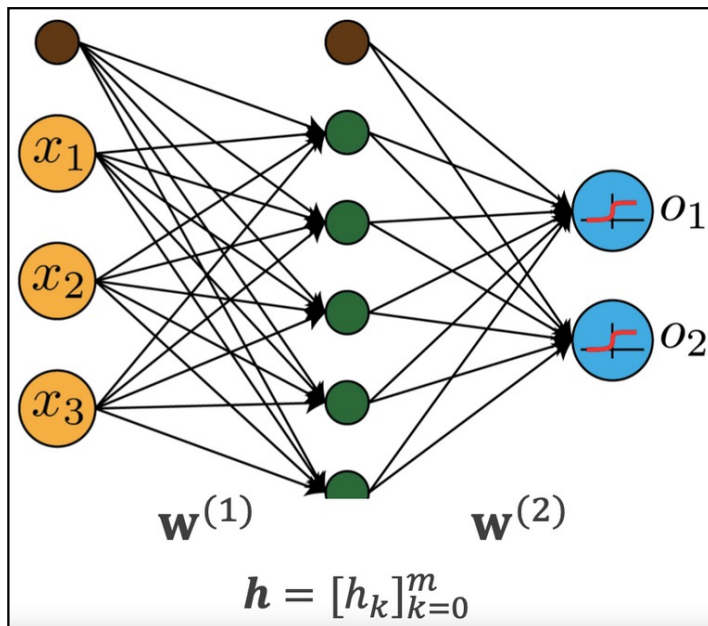
Know input component without “expected output”

distribute the error backwards through the network to the earlier layer

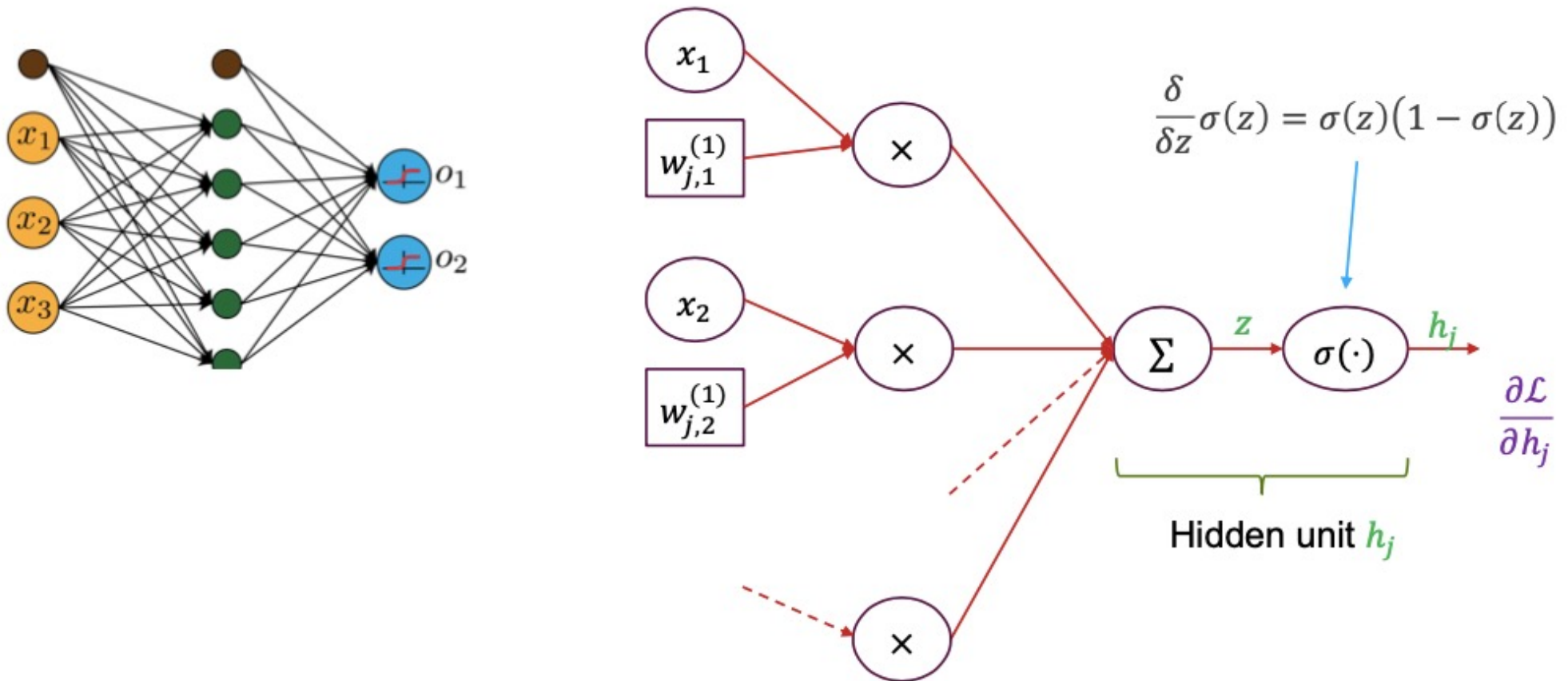
→ **back-propagation**



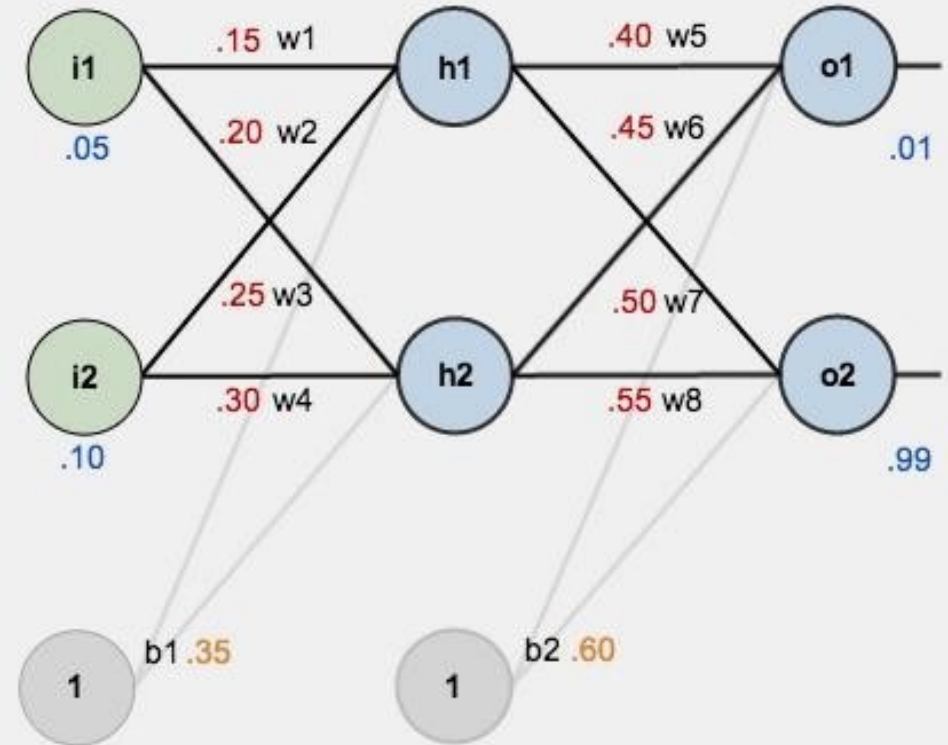
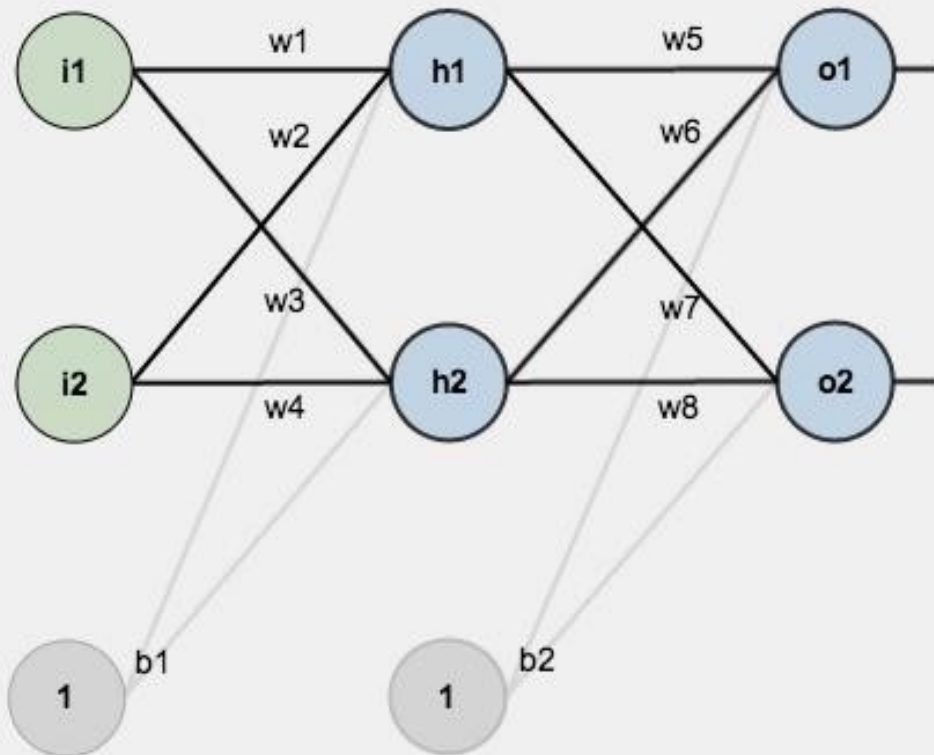
Back-Propagation



Back-Propagation via the Chain Rule



Back-Propagation via the Chain Rule



Training Process

The training algorithm (regardless of mode is):

1. Initialise all weights to small non-zero random values
2. For each training **instance (or batch) (*)**
 - a. Processes the instances feed-forward through the network,
 - b. Calculate the error of the batch
 - c. Update the network weights, with a feed-forward instance
3. Repeat until convergence

*For an MLP, as the updates are gradual, multiple passes of the data set is required. Each pass is **an epoch***

() **Online training**: weights are updated after each training instance*

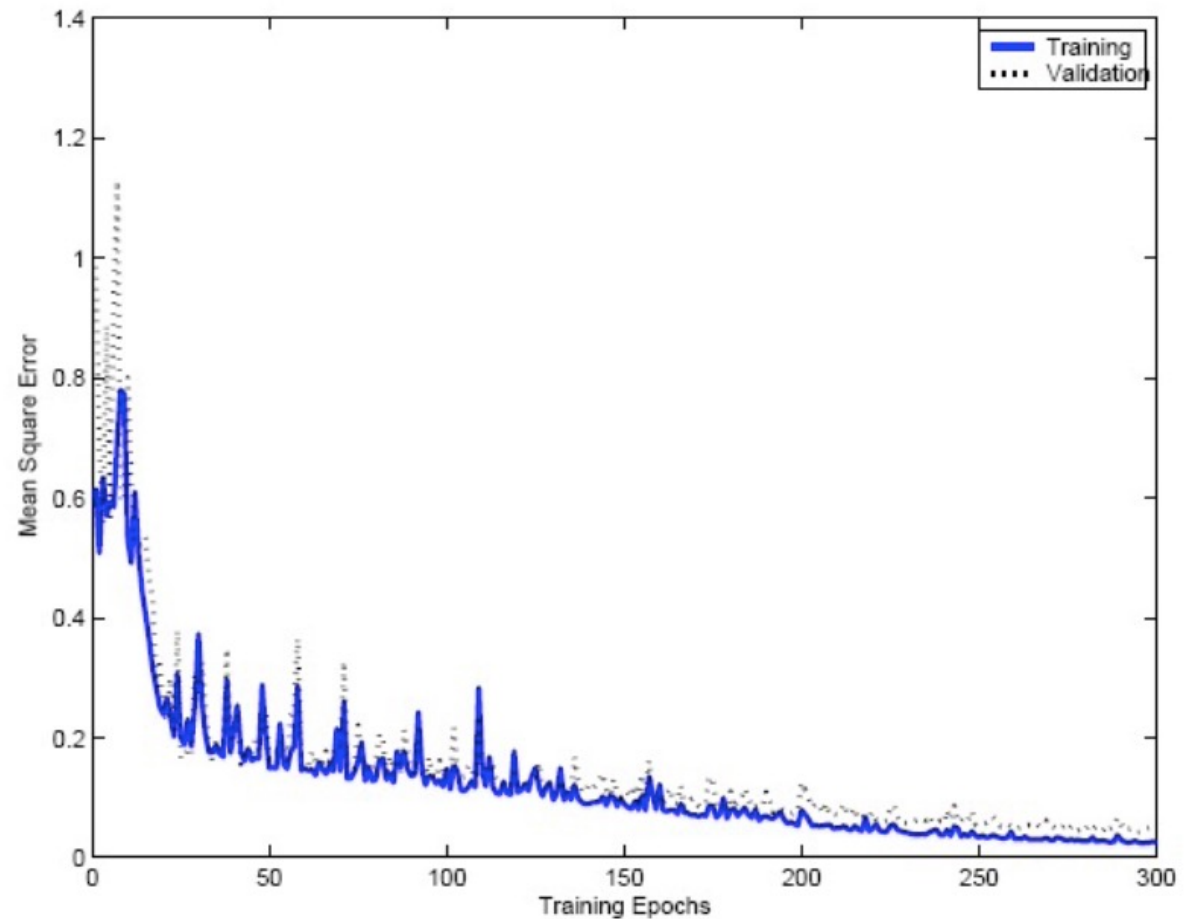
***Batch training**: error is accumulated across multiple instances (batch)*



Example training

For an MLP, as the updates are gradual, multiple passes of the data set is required.

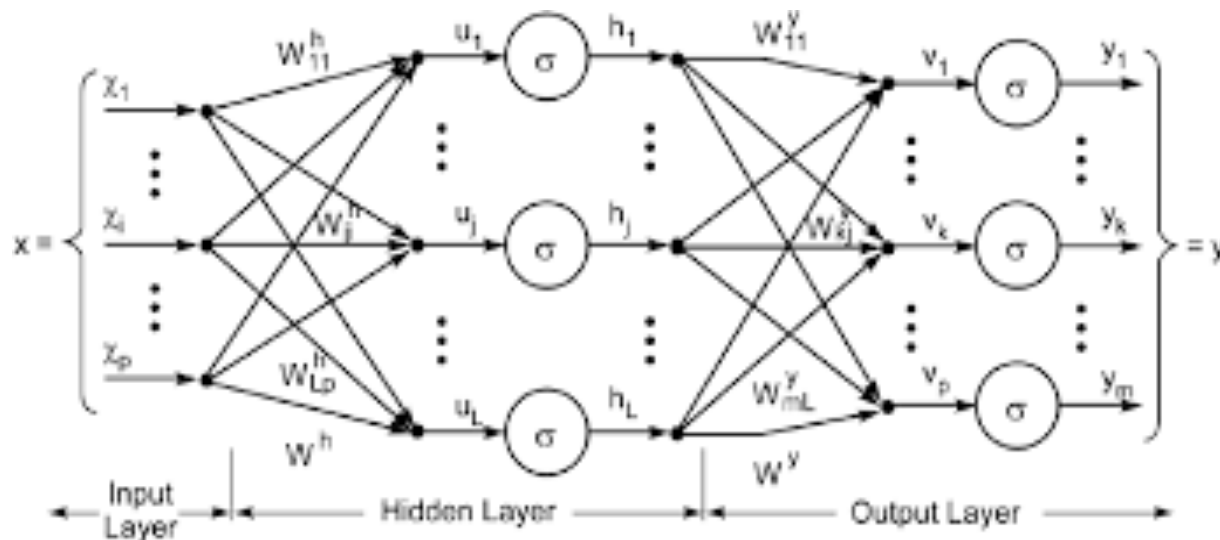
*Each pass is **an epoch***



The mean square error on training and validation sets as a function of training epochs

Strengths of MLPs

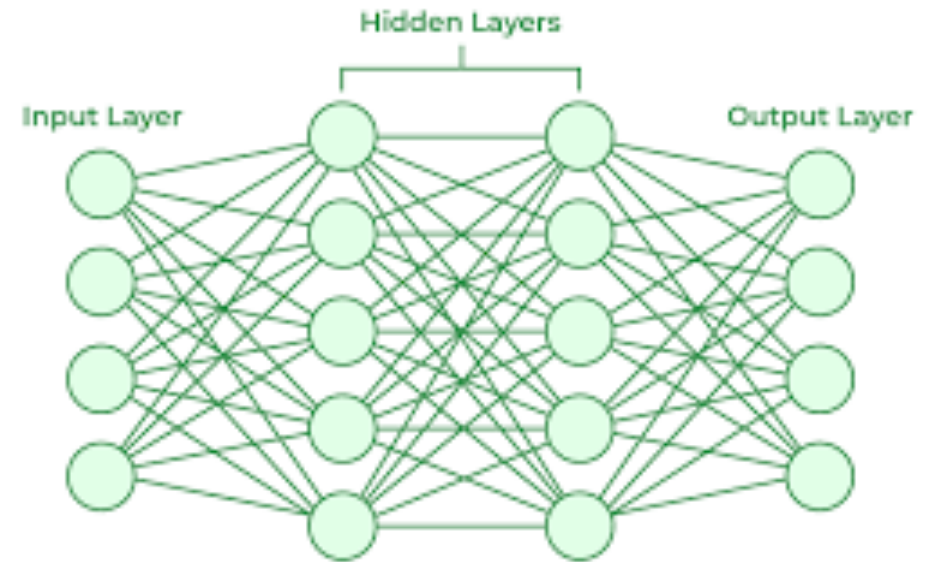
- MLPs have a number of strengths compared to other ML techniques:
 - Represent various ML problems including classification & regressions
 - Generate models with complex non-linear functions
 - “Discover” alternative features/attributes for representing a ML problem



Practical Issues

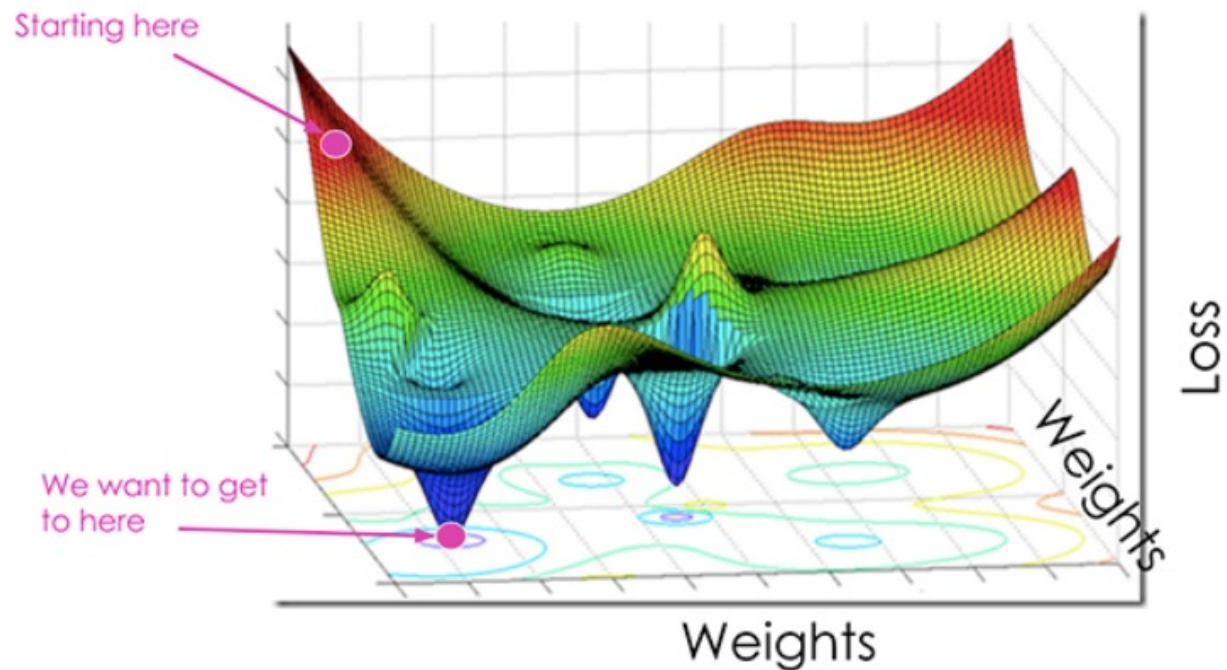
Issues in MLPs

- Convergence of back-propagation
- Network design
 - Size of the hidden layer
 - Number of free parameters
 - Network complexity
- Over-fitting



Convergence

- The back-propagation algorithm relies on gradient descent
 - Recall: gradient descent may get stuck in **local minima**



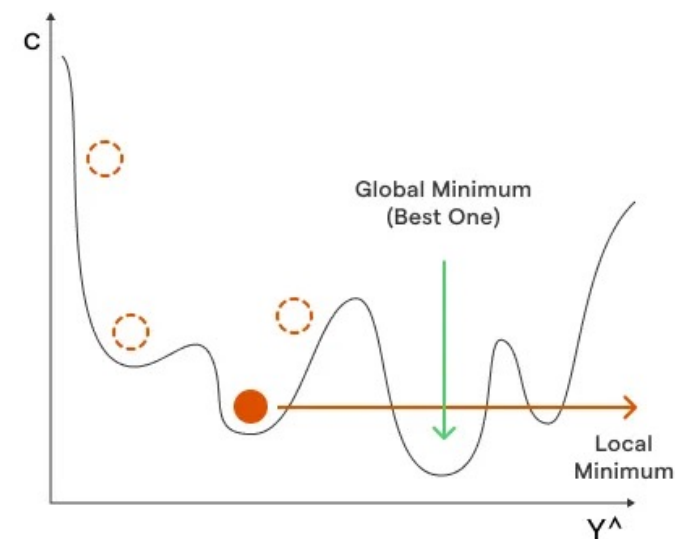
Convergence

The back-propagation algorithm relies on gradient descent

- Recall: may get stuck in local minima
- It is not guaranteed to **converge** to the optimal minimum error
- However, in practice, MLPs do converge to a suitable error
 - The **noisy** gradients via **stochastic gradient calculation** can help

$$E(\mathbf{w}) \equiv \frac{1}{b} \sum_{\mathbf{x}^{(i)} \sim p(\mathbf{x})} \text{loss} \left(y^{(i)}, h_{\mathbf{w}}(\mathbf{x}^{(i)}) \right)$$

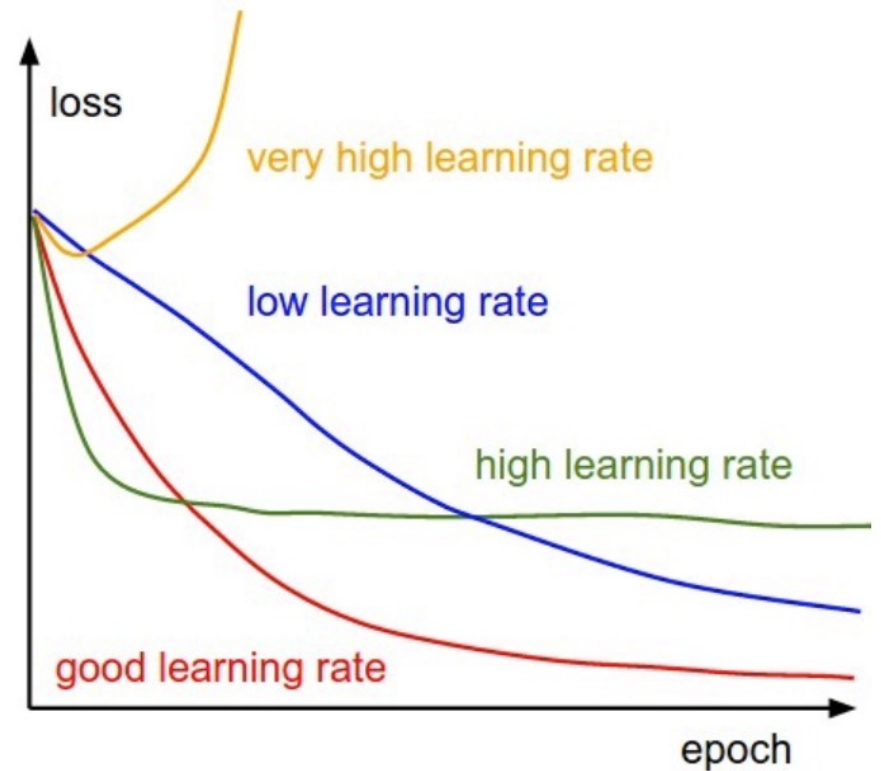
Stochastic Gradient Descent



[Stochastic gradient link](#)

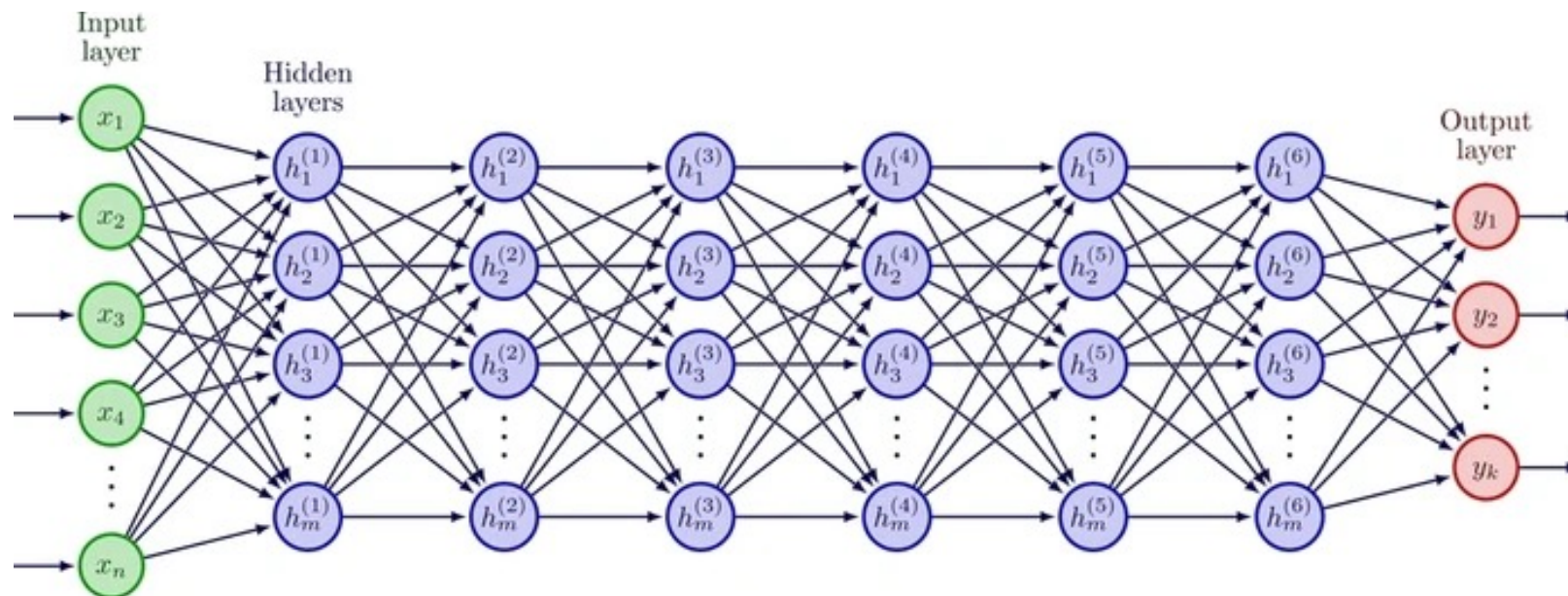
Adaptive Learning Rate

- Dynamically adjust the learning weight
- If the training error decreases: increase the learning rate to converge faster
- If the error increases: slow down learning



Network Design

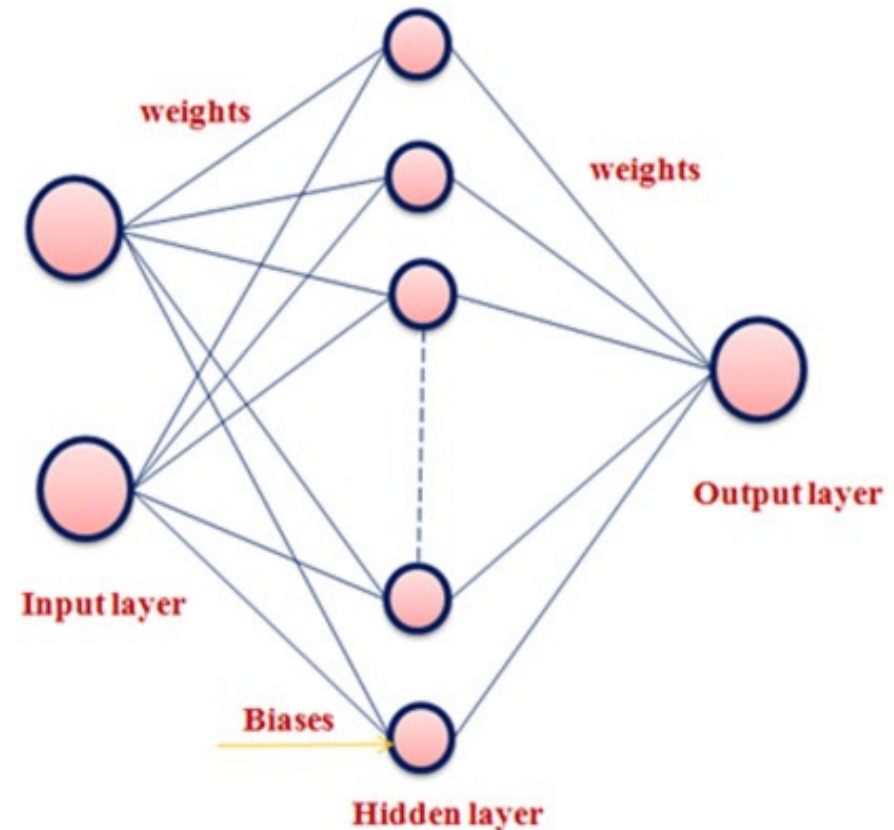
- The **number of nodes** in the hidden layer
- The form of the target output
 - Classification problem(s) - with the encoding of the classification
 - Regression problem(s) - number of output nodes and the expected value



Network Design

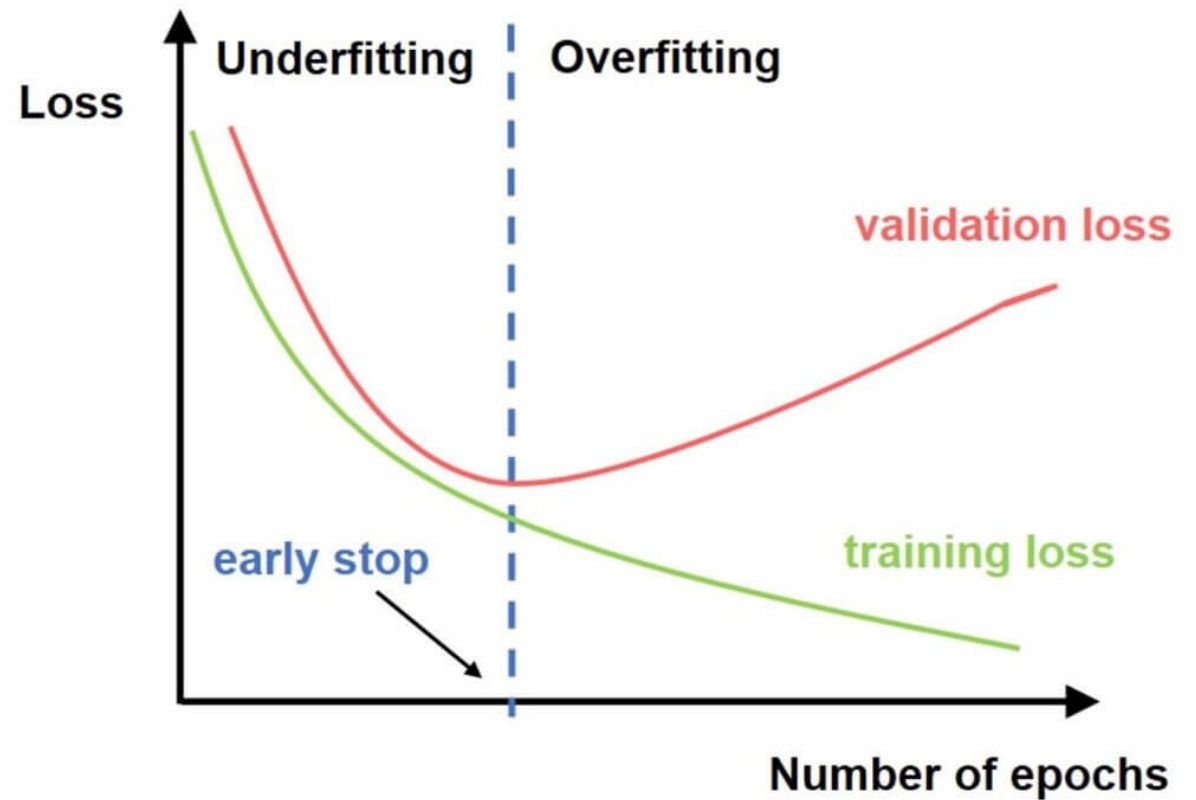
Adding more nodes:

- Makes the network more complex and **improves the internal representation**
- Potentially requires more training time
- Distributes the error(s) across more nodes
- Can also make a network too complex, leading to **high variance and overfitting**
- For too many weights, the network will not generalize well



Over-fitting

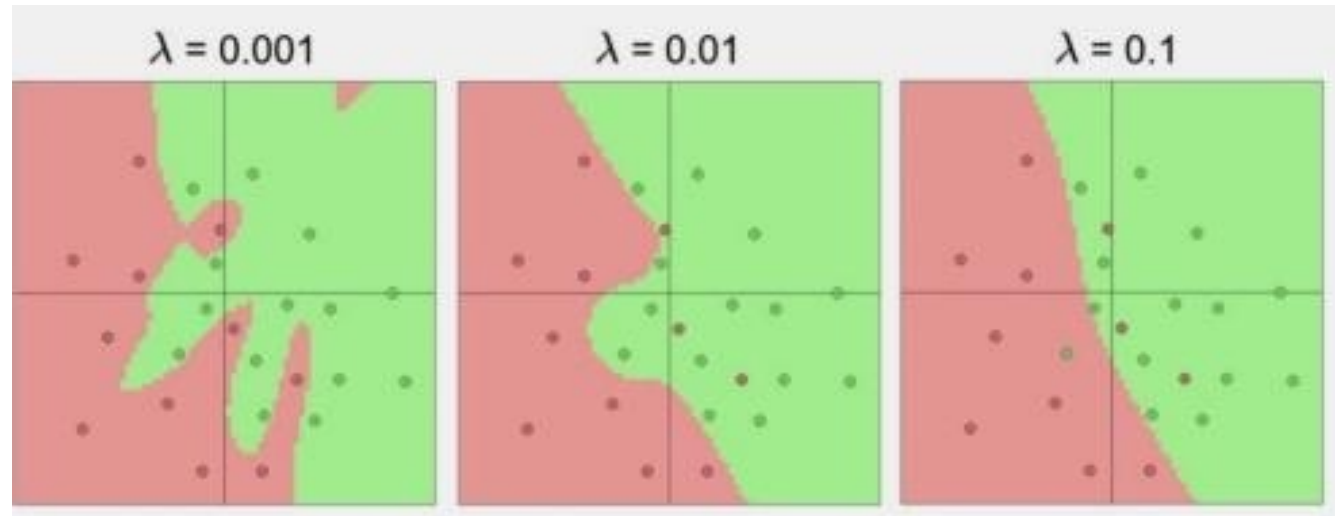
- **Early stopping**
- Regularization
- Dropout



Over-fitting

Regularization

- Lasso penalty – $\sum |w_{i,j}|$
- Ridge penalty – $\sum |w_{i,j}|^2$



Demo: <https://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>

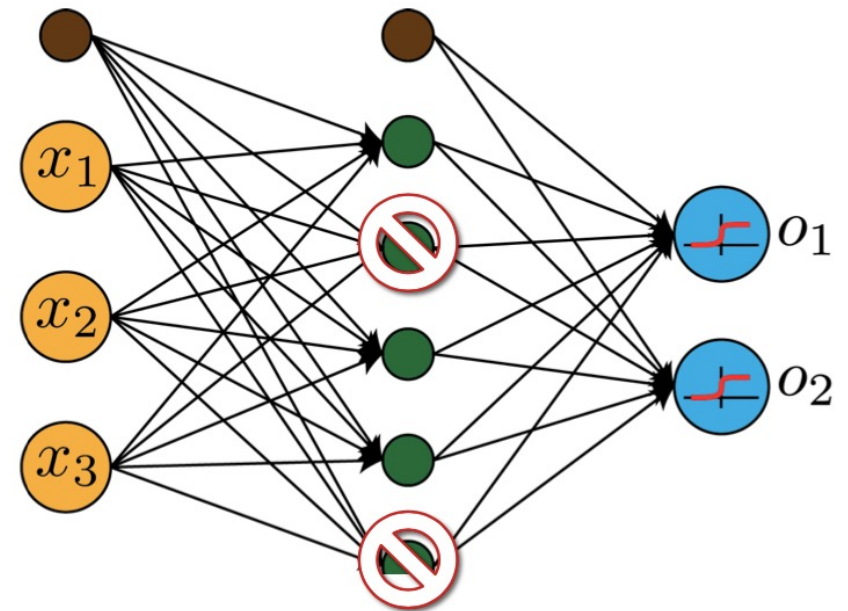
Dropout

A fully connected MLP is **symmetric**

- Each perceptron of the network is identical
- Back-propagation: symmetry, slower and less accurate in practice

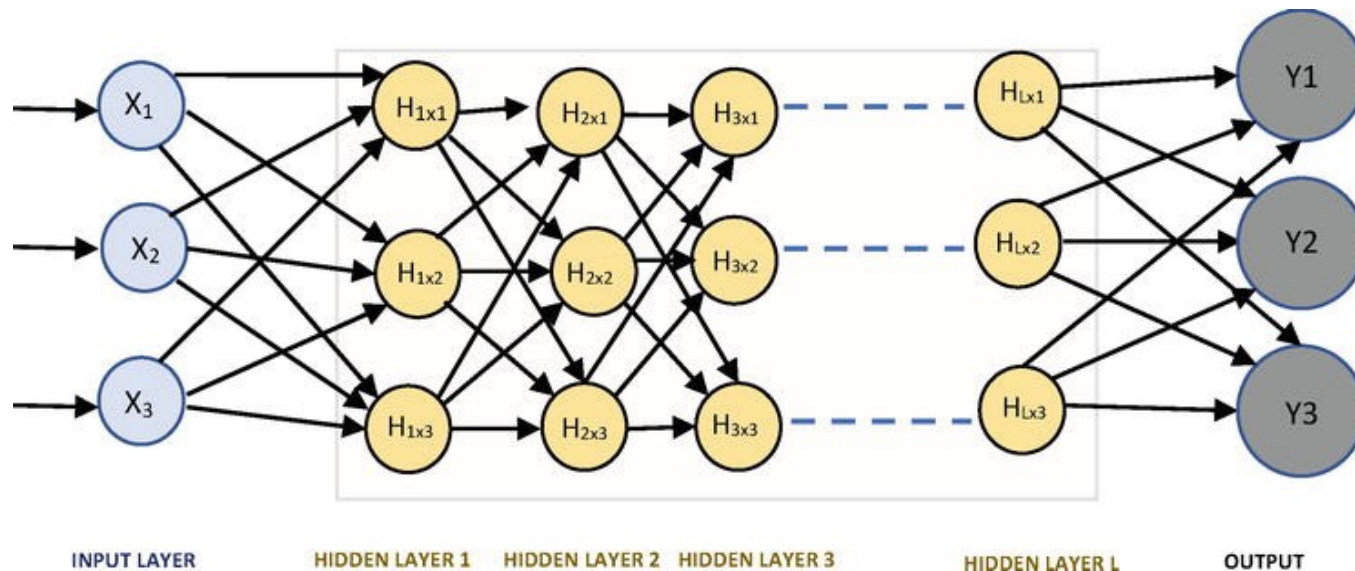
Dropout randomly removes synapses (connections) between perceptrons in MLP

- Forces a non-symmetric network
- Forces the network to consider different combinations of the input attributes
- Cannot be over reliant on a single unit

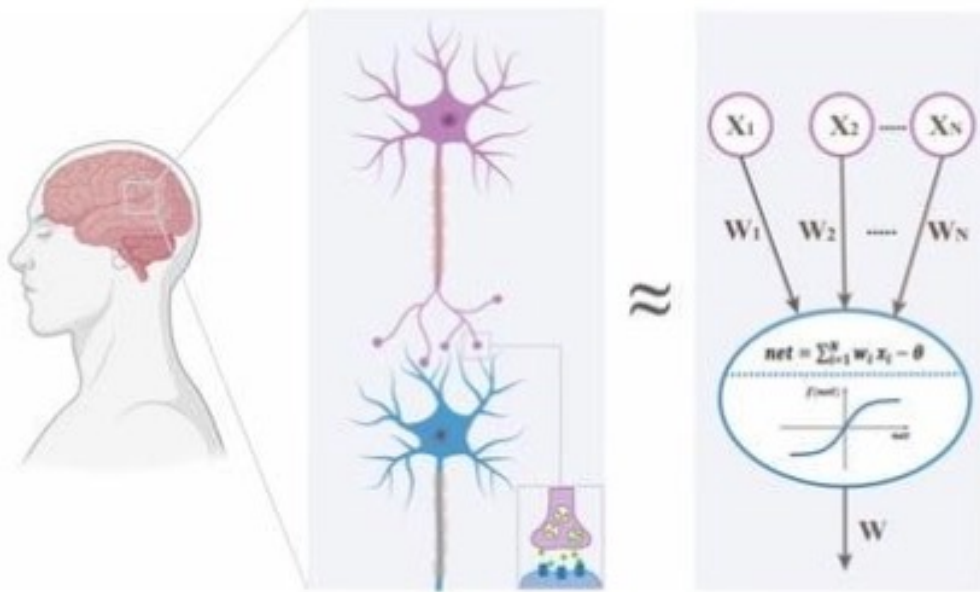


Multiple Hidden Layers

- Adding hidden layers: increase the complexity of a MLPs, can learn **at a faster rate** than adding additional perceptrons into a single hidden layer
- Produces a simpler & smaller network that has fewer weights to train
- Back propagation chain rule: errors become more diluted as layers are added



Summary



Perceptrons

- Processing & Training
- Multiple perceptrons

Multi-Layer Perceptrons (MLP)

- Feed-Forward
- Training via Back-propagation
- Strengths & Weaknesses

Practical issues

Thank you!